Overload Protection

為什麼過載是系統設計的核心問題

大多數系統在設計時，都是針對「預期的正常流量」建構的。但現實裡，流量不是平滑的曲線，它有尖峰、有突發、有惡意攻擊、有節慶促銷、有某個大 KOL 突然分享了你的連結。

當流量超過系統能承受的上限，會發生什麼？

如果你沒有任何保護機制，答案通常是：**雪崩**。請求堆積在佇列裡，延遲開始上升，執行緒池被耗盡，記憶體開始膨脹，GC 開始瘋狂，資料庫連線池塞滿，下游服務也跟著拖慢，最終整個系統停止回應。不只是超出容量的那部分請求失敗，**所有請求都失敗了**。

過載保護的核心思想是：**讓系統在壓力下優雅降級，而不是全面崩潰**。一個設計良好的系統，在 10 倍正常流量下，應該能服務其中的 30%，而不是讓 100% 的請求都失敗。

這篇講義把過載保護的工具拆解成幾個層次：

- **預防**：在入口就把過多的流量擋掉（Rate Limiting、Concurrency Limiting）
- **吸收**：用佇列把瞬間的流量峰值平滑掉（Queue-based Load Leveling）
- **擴容**：從根本上增加容量跟上需求（Auto-scaling）
- **自保**：當上面的手段都跟不上時，主動丟棄部分請求（Load Shedding、Request Prioritization）
- **隔離**：確保一個部分的過載不會波及整個系統（Bulkhead）
- **協同**：讓整條鏈路一起感知壓力（Backpressure）

Rate Limiting（速率限制）

Rate limiting 是最基本的入口防線：**限制每個客戶端在單位時間內能發出的請求數量**。

它解決兩個問題：一是防止單一客戶端（或惡意攻擊者）耗盡整個系統的資源；二是確保不同客戶端之間的公平使用。

常見的演算法

固定視窗（Fixed Window）：最簡單。每分鐘重置計數器，超過上限就拒絕。問題是在視窗邊界會有突發：一個客戶端可以在第 59 秒發 100 個請求，第 61 秒再發 100 個請求，視窗邊界的 2 秒內實際上通過了 200 個請求。

buildmoat.org

滑動視窗（Sliding Window）：用一個滾動的時間窗口來計算請求數，而不是固定的重置點。邊界突發問題消失了，但儲存每個請求的時間戳記需要更多記憶體。

令牌桶（Token Bucket）：系統以固定速率（例如每秒 10 個）往桶裡放令牌，桶有上限（例如 100 個）。每個請求消耗一個令牌，桶空了就拒絕。**優點是允許短暫的突發**，如果過去一段時間沒什麼請求，桶裡積累了令牌，可以處理一波突發。這是最符合真實場景的演算法。

漏桶（Leaky Bucket）：請求以任意速率進入，但以固定速率流出處理。強制輸出速率的平滑，不允許突發。適合需要嚴格控制處理速率的場景（例如對外部 API 的調用）。

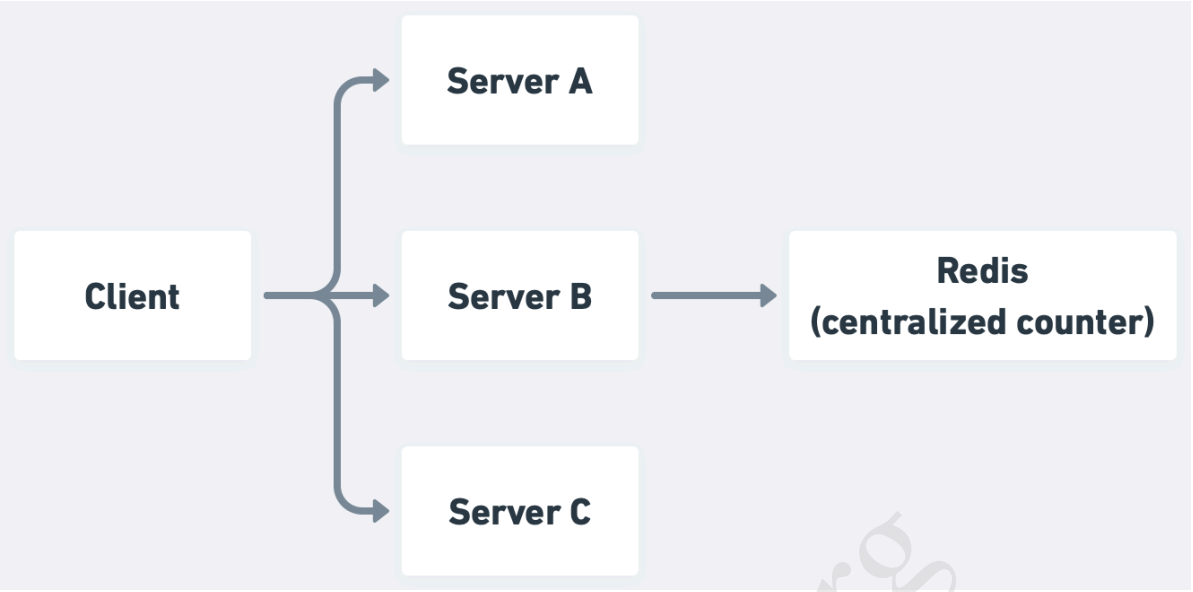
```
import time
import redis

def is_allowed(user_id: str, limit: int = 100, window: int
= 60) -> bool:
    """滑動視窗 rate limiting，使用 Redis 實現"""
    r = redis.Redis()
    key = f"rate:{user_id}"
    now = time.time()
    window_start = now - window

    pipe = r.pipeline()
    # 移除視窗之外的舊記錄
    pipe.zremrangebyscore(key, 0, window_start)
    # 計算視窗內的請求數
    pipe.zcard(key)
    # 加入這次請求的時間戳記
    pipe.zadd(key, {str(now): now})
    pipe.expire(key, window)
    results = pipe.execute()

    request_count = results[1]
    return request_count < limit
```

分散式 Rate Limiting

單機的 rate limiting 很容易實現，但在有多台伺服器的情況下，每台伺服器各自計數就沒有意義了，客戶端只需要把請求分散打到不同伺服器，就能輕鬆繞過限制。

solidnoat.org

解法是用**集中式的計數器**，通常是 Redis：



每個伺服器在處理請求前，先去 Redis 查詢並更新計數器。Redis 的原子操作（INCR + EXPIRE，或 Lua script）保證計數不會有 race condition。

延遲的代價：每個請求多了一次 Redis 查詢，通常在 1ms 以內，可以接受。如果需要極低延遲，可以用本地計數器 + 定期同步的方案，犧牲一點精確性換取速度。

Rate Limiting 的回應

被 rate limit 的請求應該回傳 `429 Too Many Requests`，並在 `Retry-After` header 裡告知客戶端多久之後可以重試：

```
HTTP/1.1 429 Too Many Requests
Retry-After: 30
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1700000060
```

這讓客戶端能夠配合退避，而不是盲目重試，進一步加重伺服器的壓力。

Concurrency Limiting（並發限制）

Rate limiting 限制的是**請求速率**（每秒幾個）。Concurrency limiting 限制的是**同時在處理中的請求數量**。

這兩者的區別很重要。想像一個請求平均需要 500ms 處理。如果你的系統有 100 個執行緒，最大的安全並發數就是 100，只要超過這個數字，新請求就只能等待，執行緒池

buildmoat.org

被耗盡，延遲開始雪崩。

Rate limiting 說的是「每秒最多 200 個請求」，但如果每個請求要花 1 秒，同時就有 200 個在跑，遠超過執行緒池的容量。

Concurrency limiting 更直接地對應到系統資源：

```
import threading

class ConcurrencyLimiter:
    def __init__(self, max_concurrent: int):
        self.semaphore = threading.Semaphore(max_concurren
t)

    def __enter__(self):
        acquired = self.semaphore.acquire(blocking=False)
        if not acquired:
            raise OverloadError("Too many concurrent reques
ts")
        return self

    def __exit__(self, *args):
        self.semaphore.release()

limiter = ConcurrencyLimiter(max_concurrent=50)

def handle_request(request):
    with limiter:
        return process(request)
```

Concurrency limiting 和 rate limiting 通常一起使用：rate limiting 擋突發流量，concurrency limiting 保護內部資源不被耗盡。

Adaptive Concurrency Limiting：Netflix 開源的 concurrency limiter 能根據延遲動態調整並發上限。當延遲升高（代表系統開始過載），自動降低並發數；延遲下降時，再逐步放寬。這比靜態設定的值更能適應真實的系統狀況。

Queue-based Load Leveling（佇列削峰）

Rate limiting 和 concurrency limiting 都是**拒絕**多餘的請求。但有時候，你不想拒絕，你只是需要時間處理。

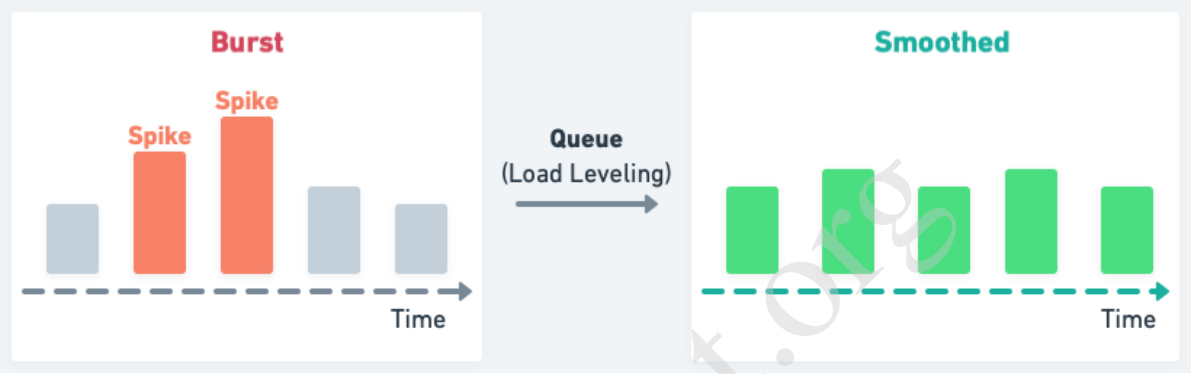
buildnoat.org

佇列削峰的思路是在生產者和消費者之間放一個訊息佇列。前端把請求快速接收進佇列，後端以自己能承受的速率從佇列消費並處理。

```
用戶請求 → API → 訊息佇列（Kafka / SQS）→ 工作服務（慢慢處理）
```

這對**非同步任務**特別有效：影片轉碼、電子郵件發送、報表生成、訂單確認，這些操作不需要在用戶等待的那幾秒內完成，完全可以放進佇列，稍後處理。

削峰的效果：



佇列把瞬間的尖峰吸收進去，後端看到的是平滑的、自己能承受的流量。

代價：請求的處理從同步變成了非同步，用戶拿到的是「你的請求已收到，正在處理」而不是立即的結果。不是所有場景都能接受這個模式，用戶支付完後等 30 秒才收到確認不太合理，但影片上傳後等幾分鐘轉碼完成是正常的。

佇列深度監控：如果佇列積壓的訊息數量持續增長，代表消費速率跟不上生產速率。這是 auto-scaling 的觸發訊號，也可能代表需要更多的消費者或更快的後端。

buildmoat.org

Auto-scaling（自動擴縮容）

前面幾個手段都是在現有容量內管理流量。Auto-scaling 是從根本上增加容量，讓系統能在需求增加時橫向擴展，需求減少時縮回來節省成本。

觸發指標

Auto-scaling 需要一個觸發條件來決定什麼時候要擴容。常見的指標：

- **CPU 使用率**：超過 70% 就擴容，是最常見的設定，但不一定是最準確的，一個 I/O 密集的服務 CPU 使用率低，但可能已經過載
- **Request per second（RPS）**：直接反映流量，更準確
- **佇列深度**：消費者服務根據佇列積壓量決定是否需要更多實例

- **P99 延遲**：當 P99 超過閾值，代表系統開始承壓，是最能反映用戶體驗的指標

```
# AWS Auto Scaling 設定範例
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-service
spec:
  minReplicas: 2
  maxReplicas: 50
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  - type: Pods
    pods:
      metric:
        name: requests_per_second
      target:
        type: AverageValue
        averageValue: "1000"
```

buildmoat.org

Reactive vs Predictive

Reactive（反應式）：流量上來了，CPU 升高了，才觸發擴容。問題是擴容需要時間，啟動新的容器實例、通過健康檢查、加入負載平衡池，通常需要 1 到 3 分鐘。在這段時間內，過載仍然存在，需要其他手段（load shedding）撐住。

Predictive（預測式）：根據歷史流量模式，提前擴容。電商平台知道每天早上 9 點流量會上來，可以在 8:50 就開始擴容，而不是等到流量真的來了才反應。AWS、GCP 都提供基於機器學習的預測性擴容功能。

Reactive + Predictive 搭配使用：預測式處理已知的流量模式，Reactive 處理預期之外的突發。

Scale Down 的謹慎性

擴容要快，縮容要慢。如果縮容太激進，一個小的流量回升就又要觸發擴容，產生振盪（oscillation）。通常設定：擴容條件觸發 1 分鐘後執行，縮容條件需持續 10 分鐘才執行。

Load Shedding（負載卸除）

Auto-scaling 需要時間，佇列也有深度上限，rate limiting 只擋外部攻擊。當這些手段都跟不上，系統仍然面對超出容量的流量時，最後的防線是 **load shedding**，主動丟棄部分請求，保護系統的核心功能不崩潰。

聽起來很極端，但考慮兩個選項：

- 處理 30% 的請求，其他 70% 快速失敗並收到明確的錯誤
- 試圖處理 100% 的請求，結果系統崩潰，100% 的請求都失敗

Load shedding 選擇前者。讓部分請求快速失敗，遠比讓所有請求一起慢死要好。

在哪裡做 Load Shedding

在入口（API Gateway / Load Balancer）：在請求進入系統之前就丟棄。成本最低，幾乎不消耗後端資源。但決策資訊少，只能基於基本的指標（請求速率、連線數）。

在服務內部：在服務開始處理之前，先檢查系統狀態（CPU、記憶體、延遲）。如果超過閾值，立即回傳 `503 Service Unavailable`，不執行任何業務邏輯。決策更精確，但請求已經進來了，消耗了一些資源。

```
import psutil

def check_system_health() -> bool:
    """檢查系統是否健康，決定是否要 shed load"""
    cpu_usage = psutil.cpu_percent(interval=0.1)
    memory_usage = psutil.virtual_memory().percent

    if cpu_usage > 85 or memory_usage > 90:
        return False  # 系統過載，開始 shed load
    return True

def handle_request(request):
    if not check_system_health():
        return Response(status=503, body="Service temporari
```

buildnow.org

```
ly unavailable")
    return process(request)
```

什麼請求應該被丟棄

不是隨機丟棄，而是有策略地丟棄：

- **重試請求優先丟棄**：如果請求帶有重試標記（`X-Retry-Count: 2`），代表它已經讓系統處理過了，這次過載可能就是它造成的。優先丟棄重試請求，保護第一次嘗試的新請求
- **低優先級請求先丟棄**：分析請求、監控上報、非關鍵的推薦系統，比用戶的核心操作可以更快丟棄
- **讀取可以比寫入更早丟棄**（視場景而定）：讀取有快取保底，寫入一旦失敗可能有資料遺失的風險

Request Prioritization（請求優先級）

Load shedding 決定「要不要丟棄」，request prioritization 決定「丟棄誰」。

在過載時，不是所有請求都是平等的：

優先級	請求類型	理由
最高	付費用戶的核心操作	直接影響收入和 SLA
高	一般用戶的核心操作	業務關鍵功能
中	非核心功能（推薦、搜尋）	可以降級或跳過
低	後台任務、分析請求	可以延遲處理
最低	監控、日誌上報	不影響用戶

實現方式通常是在請求進入服務時，根據用戶身份、請求類型等資訊打上優先級標記。過載時，低優先級的請求先被丟棄。

```
def get_priority(request) -> int:
    """決定請求的優先級（數字越小優先級越高）"""
    if request.user.is_premium:
        return 1
    if request.path in CRITICAL_PATHS:
        return 2
    if request.path in ANALYTICS_PATHS:
        return 4
```

buildmoat.org

```
    return 3

class PriorityQueue:
    def enqueue(self, request):
        priority = get_priority(request)
        self.heap.push((priority, request))

    def should_shed(self, request) -> bool:
        """當佇列超過容量，丟棄優先級最低的請求"""
        if self.size() > self.max_size:
            lowest = self.heap.peek_max()  # 優先級最低的請求
            if get_priority(request) >= lowest.priority:
                return True  # 新請求優先級不比最低的好，直接丟棄
        return False
```

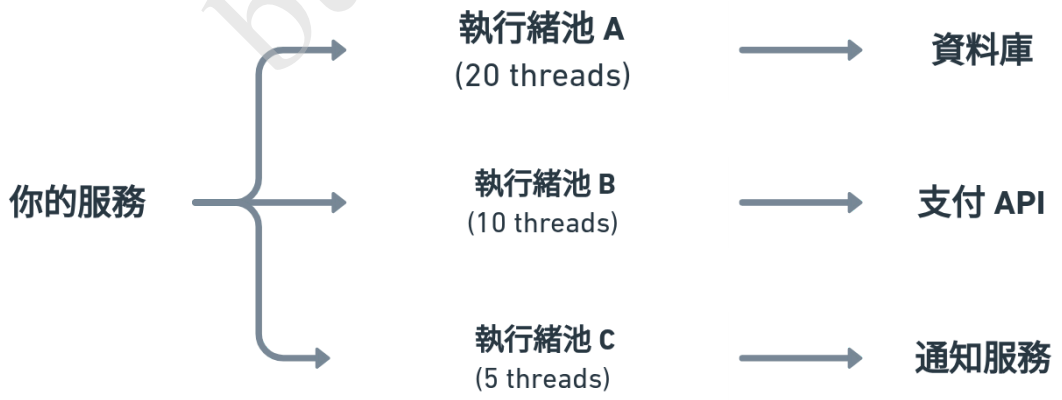
Bulkhead（艙壁隔離）

名字來自輪船的設計：把船艙分成多個隔艙，一個艙進水不會讓整艘船沉沒。

在系統設計裡，Bulkhead 的思想是：**把不同的工作負載、不同的租戶、或不同的服務依賴隔離到獨立的資源池裡**，確保一部分的過載不會耗盡整個系統的資源。

執行緒池隔離

最常見的實現：為不同的下游依賴分配獨立的執行緒池。



如果支付 API 變慢，它的執行緒池 B 被耗盡，最多損失 10 個執行緒。資料庫的執行緒池 A 完全不受影響，核心功能繼續正常運作。沒有 bulkhead 的情況下，支付 API 的慢速會把整個共用執行緒池吃光，讓資料庫操作也開始超時。

buildmeat.org

Netflix 的 Hystrix 就是以執行緒池隔離為核心設計的，每個依賴服務都有自己的執行緒池和熔斷器。

租戶隔離

在多租戶 SaaS 系統裡，一個大客戶的突發流量不應該影響其他客戶的服務品質：

```
租戶 A（大企業）   → 獨立的服務實例 / 資料庫分片
租戶 B（中型公司）→ 共用服務實例（但有 concurrency limiting）
租戶 C（小公司）   → 共用服務實例（但有 concurrency limiting）
```

高付費的大客戶可以有獨立的資源池；小客戶共用資源，但靠 concurrency limiting 確保一個小客戶的爆量不影響其他人。

連線池隔離

資料庫連線池是另一個常見的 bulkhead 邊界。把讀取操作和寫入操作分配到不同的連線池，讓大量的讀取查詢不會耗盡寫入所需的連線：

```
# 讀寫分離的連線池
read_pool = ConnectionPool(max_connections=80, host=READ_RE
PLICA)
write_pool = ConnectionPool(max_connections=20, host=PRIMAR
Y)

def get_user(user_id):
    with read_pool.get_connection() as conn:  # 讀取走 repli
ca
        return conn.query(...)

def update_user(user_id, data):
    with write_pool.get_connection() as conn:  # 寫入走 prim
ary
        return conn.execute(...)
```

Backpressure（背壓）

前面所有的機制，都是讓「承壓的那一側」自己做決定。Backpressure 不同，它讓**下游主動告知上游「我撐不住了，慢一點」**，讓壓力訊號沿著鏈路向上傳播。

buildmoat.org

沒有 backpressure 的系統裡，上游只會不斷地把請求打向下游，佇列無限增長，最終崩潰。有了 backpressure，上游在下游發出訊號後主動放慢速度，整條鏈路協同運作。

TCP 的 Backpressure

TCP 內建了 backpressure 機制。接收方有一個接收緩衝區（receive window），當緩衝區快滿了，TCP 會通知發送方縮小傳送視窗，讓發送方自動放慢。這就是網路層面的 backpressure。

gRPC 的 Backpressure

gRPC 基於 HTTP/2，HTTP/2 有 **flow control** 機制：

- 每個 stream 和整個 connection 都有流量控制視窗
- 接收方可以暫停接收，讓發送方等待
- gRPC 客戶端和服務端都能感知這個壓力，自然地放慢請求

```
# gRPC streaming 中，服務端透過處理速度自然施加 backpressure
async def process_stream(request_iterator):
    async for request in request_iterator:
        # 如果這裡處理很慢，gRPC 的 flow control 會讓
        # 客戶端自動放慢發送速率
        result = await slow_processing(request)
        yield result
```

應用層 Backpressure

除了協議層的 backpressure，應用層也可以主動實現：

```
# 服務端透過回應 429 + Retry-After，讓客戶端主動退避
def handle_request(request):
    queue_depth = message_queue.depth()

    if queue_depth > BACKPRESSURE_THRESHOLD:
        # 告知客戶端放慢速度
        return Response(
            status=429,
            headers={"Retry-After": "5"},
            body="Server busy, please slow down"
```

buildmoat.org

```
        )

    return process(request)
```

Backpressure 和 Load Shedding 的差別：load shedding 是丟棄請求；backpressure 是讓上游放慢，不丟棄任何東西，只是讓整條管線的流速降下來。Backpressure 更溫和，但需要上游配合（支持 backpressure 訊號），不是所有情況都能用。

這些工具如何組合使用

過載保護不是選一個工具，而是多層次的組合：

```
外部流量進來
      ↓
Rate Limiting（擋惡意請求和超量的單一客戶端）
      ↓
Concurrency Limiting（保護後端資源不被耗盡）
      ↓
Queue-based Load Leveling（吸收突發，平滑流量）
      ↓  ← Auto-scaling 持續在這一層擴充容量
後端服務處理
      ↓
Bulkhead（隔離不同依賴，防止局部過載蔓延）
      ↓
Load Shedding + Request Prioritization（最後防線：選擇性丟棄低優先級請求）
      ↑
Backpressure（整條鏈路的協同減速訊號）
```

在面試中，不需要把每一層都列出來。重要的是理解**為什麼**需要多層保護：每一層都有它能擋住的東西，也有它擋不住的東西。

什麼時候在面試裡用這些

主動說明過載策略

在設計任何有大流量的系統時，主動說明你的過載保護策略是加分項：

buildmoat.org

「我們會在 API Gateway 做 rate limiting，每個用戶每分鐘最多 1000 個請求，用 Redis + 滑動視窗實現。核心的訂單處理服務設定 concurrency limiting，最多 50 個並發請求，超過這個數字，新請求立刻回傳 503，讓客戶端退避重試。對於不需要實時處理的任務（通知、報表），放進 SQS，讓後端以自己的速率慢慢消費，這樣即使流量尖峰，後端也不會崩潰。」

常見面試情境

設計電商秒殺系統：「秒殺活動的流量可能是平常的 100 倍。在商品詳情頁我會用 CDN + 靜態快取（fallback）讓 99% 的讀取請求不打到後端。下單請求進入 Redis 隊列，Lua script 做原子性的庫存扣減，超賣直接回傳失敗。後端以固定速率從隊列取單處理，完全不受前端流量影響。同時配合 rate limiting 防止同一用戶的重複提交。」

設計多租戶 SaaS：「我會對每個租戶設定獨立的 rate limit 配額。高付費的企業客戶有獨立的服務實例（bulkhead），中小客戶共用實例但各自有 concurrency limit，確保一個客戶的突發流量不影響其他人。」

設計視訊轉碼服務：「用戶上傳影片後，任務進入 SQS 佇列。我根據佇列深度來觸發 Auto-scaling 擴充轉碼 worker 的數量，佇列深度超過 100 就加一台，低於 10 持續 10 分鐘就縮一台。這樣高峰時自動擴容，閒置時縮回來節省成本。」

常見的 Deep Dive 問題

「Rate limiting 和 Load shedding 有什麼區別？」

「Rate limiting 是針對**個別客戶端**的保護，限制某個特定用戶或 IP 能發出多少請求。它解決的是公平性和防濫用問題。即使沒有任何單一客戶端違規，總流量還是可能超過系統容量。Load shedding 是針對**整個系統狀態**的保護，不管流量從哪裡來，當系統整體過載時，主動丟棄部分請求。兩者是互補的：rate limiting 是入口的公平性控制，load shedding 是系統的最後防線。」

「Auto-scaling 啟動需要時間，在這段時間怎麼辦？」

「Reactive auto-scaling 通常需要 1 到 3 分鐘，這段時間需要其他手段撐住。一是**預測性擴容**，根據歷史流量提前擴好；二是**load shedding 兜底**，在 auto-scaling 跟上之前，系統以較低的容量服務，優先處理高優先級請求，丟棄低優先級的；三是**佇列緩衝**，非同步任務進佇列等待，auto-scaling 跟上後再處理積壓。這幾個機制配合，讓系統在擴容期間能夠存活。」

「怎麼決定 rate limit 的閾值？」

「要根據兩個數字來算：後端的實際容量，以及正常用戶的合理使用量。如果後端每秒能處理 10,000 個請求，有 1,000 個活躍用戶，每個用戶的合理 rate limit 大約是 10

buildmoat.org

req/s，但你要留 buffer，可以設 20 req/s 以應對偶發的合理突發。Rate limit 設得太低會傷害正常用戶體驗，太高就失去保護意義。另外，不同端點應該有不同的 limit，搜尋 API 可以比較寬鬆，登入端點要嚴格（防止暴力破解），管理員 API 需要更嚴格的驗證而不只是速率限制。」

「Bulkhead 的粒度怎麼決定？」

「粒度太細，管理成本太高，資源也容易浪費（一個執行緒池閒著，另一個卻排隊）；粒度太粗，隔離就沒有意義。實務上的建議是：根據**故障影響的相關性**來劃分。把那些『如果它掛掉，你希望不影響哪些功能』的依賴隔離成獨立的 bulkhead。例如支付 API 掛掉不應該影響商品瀏覽，所以它們應該是獨立的 bulkhead。同一個 bulkhead 內的依賴，故障時可以一起降級。從 3 到 5 個 bulkhead 開始，根據實際的故障模式再調整。」

總結

過載保護的核心思想是：**讓系統在超出設計容量時，有節制地降級，而不是全面崩潰**。

- **Rate Limiting**：在入口按客戶端限制速率，解決公平性和防濫用
- **Concurrency Limiting**：限制同時處理中的請求數，直接保護後端資源
- **Queue-based Load Leveling**：用佇列吸收流量峰值，讓後端以自己的節奏處理
- **Auto-scaling**：動態擴充容量，從根本上跟上需求；要快速擴容、謹慎縮容
- **Load Shedding**：當容量真的跟不上時，主動丟棄部分請求，讓系統能繼續服務其餘的
- **Request Prioritization**：丟棄時有策略，優先保住對業務最關鍵的請求
- **Bulkhead**：隔離不同工作負載的資源，讓局部的過載不蔓延成全面崩潰
- **Backpressure**：讓壓力訊號往上游傳播，讓整條鏈路協同降速

這些工具沒有哪一個能單獨解決過載問題，它們是層層疊加的防線。在面試裡，展示你知道**為什麼**要多層保護、**每一層解決的是哪個問題**，比背出所有名詞更有說服力。

buildneat.org